Strings in Java

The Concepts

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-14

Outline

1. What Is a String?
2. Strings Are Objects
3. Immutability
4. Equality and Order
5. Characters, Unicode and Bytes
6. Summary

What Is a String?

Text is everywhere

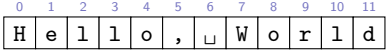
Programs handle text all the time:

- names, addresses, messages
- commands typed by a user
- file and web page contents
- DNA sequences: `GATTACA`
- source code itself

Definition

A **string** is a finite sequence of **characters**. Its **length** is the number of characters.

The string `"Hello, World"` has length 12:



Each character has a **position** (index), starting at **0**. The space and the comma are characters too.

The **empty string** `""` has length 0.

A character and a string are different things

	`char`	`String`
holds	exactly **one** character	**zero or more** characters
literal	single quotes: `'A'`	double quotes: `"A"`, `""`, `"Ali"`
kind of type	**primitive** (a 16-bit number)	**class** (objects, with methods)
stored	the value itself	a **reference** to an object
default value	`'\u0000'`	`null` (no object)
compare with	`==`	`equals(...)`

`'A'` is not `"A"`

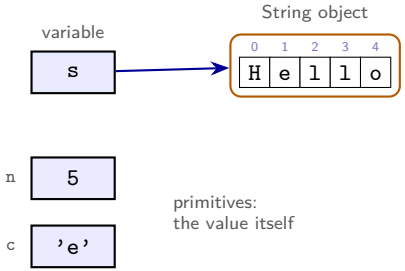
`'A'` is the number 65 in disguise. `"A"` is an object that *contains* one character. Mixing them up is one of the most common beginner errors.

Strings Are Objects

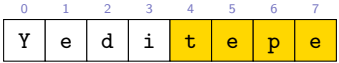
A variable holds a reference

```
String s = "Hello";
int    n = s.length();    // 5
char   c = s.charAt(1);   // 'e'
```

- `s` does not contain the letters; it **refers** (points) to a `String` object
- We **ask the object**: `s.length()`, `s.charAt(1)`
- The object knows its characters and offers about 70 methods



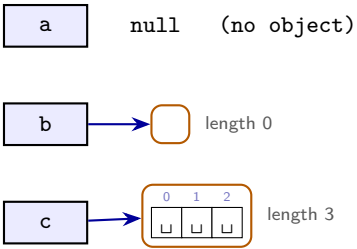
Positions start at 0



Expression	Value	Why
`s.length()`	8	8 characters
`s.charAt(0)`	`'Y'`	first index is 0
`s.charAt(s.length()-1)`	`'e'`	last index is `length-1`
`s.substring(4, 8)`	`"tepe"`	from 4 **up to, not including**, 8 (highlighted)
`s.indexOf("tepe")`	4	where it starts; `-1` if not found
`s.charAt(8)`	*exception*	index 8 does not exist

Rule of thumb: `substring(b, e)` has length `e - b`.

Nothing, empty, or blank?




```
String a = null;
String b = "";
String c = "   ";
```

	a	b	c
`x == null`	true	false	false
`x.isEmpty()`	*error*	true	false
`x.isBlank()`	*error*	true	true

NullPointerException

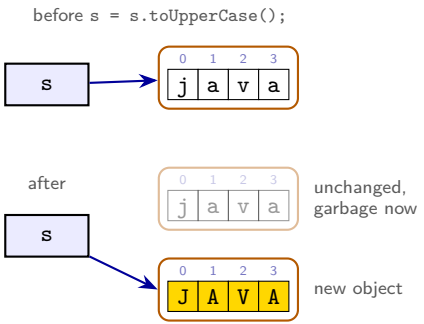
Calling a method on `null` fails: there is no object to ask. Check `x != null` first, or write `"text".equals(x)`.

Immutability

A String never changes

```
String s = "java";
s.toUpperCase();       // result lost!
s = s.toUpperCase();   // s now refers
                       // to a new object
```

- Every “changing” method (`toUpperCase`, `replace`, `trim`, ...) **returns a new** `String`
- The original object is never modified
- An object nobody refers to is removed by the **garbage collector**



Why are strings immutable?

Safe sharing

Many variables can refer to the same string. Nobody can change it under the others' feet.

Security

A file name or URL checked once cannot be altered after the check.

Threads

Immutable objects can be used by many threads without locks.

Hashing

Strings are the most common `HashMap` keys. Their hash code never changes, so it is computed once and cached.

The string pool

Equal literals can share one object, saving memory (next section).

The price

Building a long string piece by piece creates many temporary objects. That is what `StringBuilder` is for.

Building a string: + versus StringBuilder

```
String s = "";
for (int i = 0; i < n; i++)
    s = s + "x";    // new object each time
```

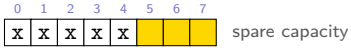
Step k copies the k characters built so far into a new object. In total:

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2} \in O(n^2)$$

For $n = 100\,000$ that is about 5 billion copied characters.

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < n; i++)
    sb.append("x"); // same buffer
String s = sb.toString();
```

A `StringBuilder` is a **mutable** buffer with spare room (capacity). When full, it doubles; the total copying stays $O(n)$.



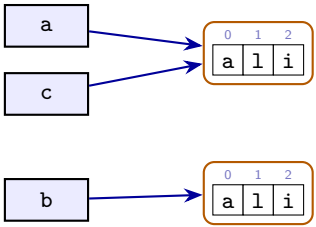
Equality and Order

Same object or same text?

```
String a = new String("ali");
String b = new String("ali");
String c = a;

a == b          // false
a.equals(b)     // true
a == c          // true
```

- == compares **references**: is it the *same object*?
- equals compares **characters**: is it the *same text*?



Rule

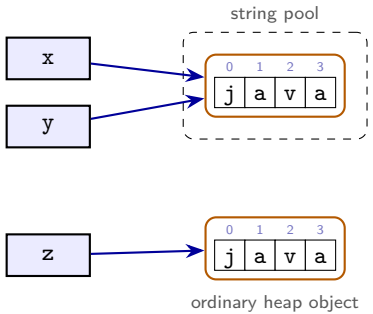
Compare strings with equals (or equalsIgnoreCase), never with ==.

The string pool

```
String x = "java";              // pool
String y = "java";              // same
String z = new String("java"); // new

x == y            // true
x == z            // false
x == z.intern()   // true
```

Literals are kept in a **pool**: equal literals share one object. `new` always makes a new object. `intern()` returns the pooled copy.



Why it still matters: `x == y` being `true` for literals makes == *look* correct in small tests, then fail on user input.

Dictionary order: `compareTo`

`a.compareTo(b)` is **negative** if a comes first, **0** if equal, **positive** if a comes after. It compares character codes from left to right.

Call	Result	Reason
`"apple".compareTo("banana")`	-1	`'a'` (97) $-$ `'b'` (98)
`"Apple".compareTo("apple")`	-32	`'A'` (65) $-$ `'a'` (97): capitals first
`"Zebra".compareTo("apple")`	-7	`'Z'` (90) $<$ `'a'` (97)
`"app".compareTo("apple")`	-2	a prefix comes first: $3-5$
`"çay".compareTo("zeytin")`	109	`'ç'` (231) $>$ `'z'` (122)

Character-code order is not the alphabet

Upper case before lower case, and Turkish letters (ç, ğ, ı, ö, ş, ü) after z. For real alphabetical order use `compareToIgnoreCase` or a `Collator` for the language.

Characters, Unicode and Bytes

Characters are numbers

Every character has a number in **Unicode**, the world-wide character table. A `char` stores this number in 16 bits.

char	decimal	hex	written
`'0'`	48	30	`'\u0030'`
`'A'`	65	41	`'\u0041'`
`'a'`	97	61	`'\u0061'`
`'ç'`	231	E7	`'\u00E7'`
`'ğ'`	287	11F	`'\u011F'`
`'ı'`	305	131	`'\u0131'`
`'İ'`	304	130	`'\u0130'`

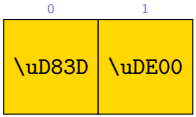
Consequences

- Arithmetic works: `'a' + 1` is 98, `(char)('a' + 1)` is `'b'`
- Digit to number: `'7' - '0'` is 7
- Letters `'a'..'z'` are consecutive, so `c - 'a'` is a position 0..25 (useful for counting letters)
- Turkish letters are **not** between `'a'` and `'z'`
- `'i'`/`'I'` and `'ı'`/`'İ'` are four different characters; upper/lower case depends on the **language** (locale)

When 16 bits are not enough

Unicode has more than a million possible **code points**, but a `char` has only 65 536 values. Characters above `U+FFFF` (most emoji, rare scripts) take **two** `chars`: a **surrogate pair**.

The emoji "grinning face", `U+1F600`:



`s.length()`	2 (chars)
`s.codePointCount(0, 2)`	1 (what a person sees)
`s.charAt(0)`	half of the emoji

Careful

`length()` counts 16-bit units, not "letters". Reversing such a string char by char breaks the pair. Use code-point methods (`codePoints()`, `codePointAt`) for such text.

Text in files: encodings

Inside Java a `String` is Unicode. A **file** or a network only has **bytes**; an **encoding** (charset) maps characters to bytes. The word `"Ağaç"`:

Encoding	Bytes (hex)	Count	
UTF-8	`41 C4 9F 61 C3 A7`	6	(1 to 4 bytes per character)
ISO-8859-9 (Turkish)	`41 F0 61 E7`	4	(1 byte, only 256 characters)
UTF-16	`00 41 01 1F 00 61 00 E7`	8	(2 bytes per `char`)

Reading UTF-8 bytes as if they were Windows-1252 gives `"AÄŸaÃ§"`: garbled text (“mojibake”). **Always state the encoding** when reading or writing text.

Windows

The console (Command Prompt) uses an old code page (857 in Turkey), so ç may print wrongly. Run `chcp 65001` to switch it to UTF-8.

macOS

Terminal uses UTF-8 by default, so Turkish letters print correctly. Since Java 18 the default file encoding is UTF-8 on both systems.

Summary

Big ideas

Idea	In one sentence
Sequence	A string is a sequence of characters, indexed from 0 to `length()-1`.
Object	A `String` variable holds a reference to an object; `null` means no object.
Immutable	A `String` never changes; methods return new strings.
Builder	Use `StringBuilder` to build a string piece by piece.
Equality	`equals` compares text, `==` compares references.
Pool	Equal literals share one object; `new String` does not.
Order	`compareTo` uses character codes, not the alphabet of a language.
Unicode	A `char` is a 16-bit number; some characters need two `char`s.
Encoding	Files hold bytes; always say which encoding (UTF-8) turns them into text.

Check yourself

1. **Q.** What is `"Yeditepe".substring(2, 5)`, and what is its length?
2. **Q.** After `String s = "ab"; s.concat("c");` what is `s`?

3. **Q.** Two strings typed by a user both contain `"yes"`. Is `a == b` true?

4. **Q.** Which comes first with `compareTo`: `"zebra"` or `"Zebra"`?
5. **Q.** Why can `"ş".getBytes(UTF_8).length` be 2 while `"ş".length()` is 1?

6. **Q.** Why is `s += x` in a long loop slow?

Check yourself

1. **Q.** What is `"Yeditepe".substring(2, 5)`, and what is its length? `"dit"`, length $5 - 2 = 3$.
2. **Q.** After `String s = "ab"; s.concat("c");` what is `s`?

3. **Q.** Two strings typed by a user both contain `"yes"`. Is `a == b` true?

4. **Q.** Which comes first with `compareTo`: `"zebra"` or `"Zebra"`?
5. **Q.** Why can `"ş".getBytes(UTF_8).length` be 2 while `"ş".length()` is 1?

6. **Q.** Why is `s += x` in a long loop slow?

Check yourself

1. **Q.** What is `"Yeditepe".substring(2, 5)`, and what is its length? `"dit"`, length $5 - 2 = 3$.
2. **Q.** After `String s = "ab"; s.concat("c");` what is `s`? Still `"ab"`: the result of `concat` was not stored.
3. **Q.** Two strings typed by a user both contain `"yes"`. Is `a == b` true?

4. **Q.** Which comes first with `compareTo`: `"zebra"` or `"Zebra"`?
5. **Q.** Why can `"ş".getBytes(UTF_8).length` be 2 while `"ş".length()` is 1?

6. **Q.** Why is `s += x` in a long loop slow?

Check yourself

1. **Q.** What is `"Yeditepe".substring(2, 5)`, and what is its length? `"dit"`, length $5 - 2 = 3$.
2. **Q.** After `String s = "ab"; s.concat("c");` what is `s`? Still `"ab"`: the result of `concat` was not stored.
3. **Q.** Two strings typed by a user both contain `"yes"`. Is `a == b` true? Usually false: they are different objects. Use `a.equals(b)`.
4. **Q.** Which comes first with `compareTo`: `"zebra"` or `"Zebra"`?
5. **Q.** Why can `"ş".getBytes(UTF_8).length` be 2 while `"ş".length()` is 1?
6. **Q.** Why is `s += x` in a long loop slow?

Check yourself

1. **Q.** What is `"Yeditepe".substring(2, 5)`, and what is its length? `"dit"`, length $5 - 2 = 3$.
2. **Q.** After `String s = "ab"; s.concat("c");` what is `s`? Still `"ab"`: the result of `concat` was not stored.
3. **Q.** Two strings typed by a user both contain `"yes"`. Is `a == b` true? Usually false: they are different objects. Use `a.equals(b)`.
4. **Q.** Which comes first with `compareTo`: `"zebra"` or `"Zebra"`? `"Zebra"`: `'Z'` (90) $<$ `'z'` (122).
5. **Q.** Why can `"ş".getBytes(UTF_8).length` be 2 while `"ş".length()` is 1?
6. **Q.** Why is `s += x` in a long loop slow?

Check yourself

1. **Q.** What is `"Yeditepe".substring(2, 5)`, and what is its length? `"dit"`, length $5 - 2 = 3$.
2. **Q.** After `String s = "ab"; s.concat("c");` what is `s`? Still `"ab"`: the result of `concat` was not stored.
3. **Q.** Two strings typed by a user both contain `"yes"`. Is `a == b` true? Usually false: they are different objects. Use `a.equals(b)`.
4. **Q.** Which comes first with `compareTo`: `"zebra"` or `"Zebra"`? `"Zebra"`: `'Z'` (90) $<$ `'z'` (122).
5. **Q.** Why can `"ş".getBytes(UTF_8).length` be 2 while `"ş".length()` is 1? `length()` counts chars; UTF-8 needs 2 bytes for ş.
6. **Q.** Why is `s += x` in a long loop slow?

Check yourself

1. **Q.** What is `"Yeditepe".substring(2, 5)`, and what is its length? `"dit"`, length $5 - 2 = 3$.
2. **Q.** After `String s = "ab"; s.concat("c");` what is `s`? Still `"ab"`: the result of `concat` was not stored.
3. **Q.** Two strings typed by a user both contain `"yes"`. Is `a == b` true? Usually false: they are different objects. Use `a.equals(b)`.
4. **Q.** Which comes first with `compareTo`: `"zebra"` or `"Zebra"`? `"Zebra"`: `'Z'` (90) $<$ `'z'` (122).
5. **Q.** Why can `"ş".getBytes(UTF_8).length` be 2 while `"ş".length()` is 1? `length()` counts chars; UTF-8 needs 2 bytes for ş.
6. **Q.** Why is `s += x` in a long loop slow? Each step copies the whole string into a new object: $O(n^2)$ in total.

Questions?